HDOC Day-71

Dynamic Matrix Operations Using Pointers and Memory Allocation in C

Problem Statement

Write a C program to dynamically allocate two matrices and perform addition, subtraction, and multiplication using pointer notation. Implement each operation in a separate function and release all allocated memory properly.

Algorithm

1. Start and read positive dimensions of matrices A and B (maximum 20 in this implementation).

2. Allocate memory for A, B and the result matrix using malloc(); check for allocation failure.

3. Read all elements using pointer arithmetic.

4. Display the menu repeatedly: Addition, Subtraction, Multiplication, Exit.

5. For addition or subtraction, require equal dimensions; compute each result using separate functions.

6. For multiplication, require columns of A = rows of B; compute each product entry with three nested loops and pointer notation.

7. Display the resulting matrix, or an error for incompatible dimensions or invalid menu choice.

8. On Exit, free all three allocations and stop.

Test Case Table

TC	Inputs / operation	Expected result
1	2x2 A: 1 2 / 3 4; B: 5 6 / 7 8; choice 1	6 8 / 10 12
2	Same matrices; choice 2	-4 -4 / -4 -4
3	Same matrices; choice 3	19 22 / 43 50
4	1x1 A: 7; B: 3; choice 1	10
5	1x1 A: 7; B: 3; choice 2	4
6	1x1 A: 7; B: 3; choice 3	21
7	A 2x3: 1 2 3 / 4 5 6; B 3x2: 7 8 / 9 10 / 11 12; choice 3	58 64 / 139 154
8	A 2x2: 1 2 / 3 4; B 2x3: 1 2 3 / 4 5 6; choice 1	Addition not possible: dimensions differ.
9	A 2x3: 1 2 3 / 4 5 6; B 2x2: 1 2 / 3 4; choice 3	Multiplication not possible.
10	1x1 A: 1; B: 1; choice 9, then 4	Invalid choice; Exiting program.

Note: Enter the matrices once per program run. Then select the listed menu choice. For Test Case 10, enter 9 followed by 4.

C Program

```
#include <stdio.h>
#include <stdlib.h>
```

```
void read_matrix(int *a, int rows, int cols) {
    int i, j;
    for (i = 0; i < rows; i++)
        for (j = 0; j < cols; j++) {
            printf("Element [%d][%d]: ", i, j);
            if (scanf("%d", a + i * cols + j) != 1) {
                printf("Invalid element.\n");
                exit(1);
            }
        }
}
void display_matrix(const int *a, int rows, int cols) {
    int i, j;
    for (i = 0; i < rows; i++) {
        for (j = 0; j < cols; j++) printf("%d ", *(a + i * cols + j));
        printf("\n");
    }
}
void add_matrices(const int *a, const int *b, int *c, int r, int col) {
    int i;
    for (i = 0; i < r * col; i++) *(c + i) = *(a + i) + *(b + i);
}
void subtract_matrices(const int *a, const int *b, int *c, int r, int col) {
    int i;
    for (i = 0; i < r * col; i++) *(c + i) = *(a + i) - *(b + i);
}
void multiply_matrices(const int *a, const int *b, int *c, int r1, int c1, int c2) {
    int i, j, k;
    for (i = 0; i < r1; i++)
        for (j = 0; j < c2; j++) {
            *(c + i * c2 + j) = 0;
            for (k = 0; k < c1; k++)
                *(c + i * c2 + j) += *(a + i * c1 + k) * *(b + k * c2 + j);
        }
}
int main(void) {
    int r1, c1, r2, c2, choice, *a, *b, *result;
    printf("Enter rows and columns of matrix A: ");
    if (scanf("%d%d", &r1, &c1) != 2 || r1 < 1 || c1 < 1 || r1 > 20 || c1 > 20) {
        printf("Invalid dimensions.\n"); return 1;
    }
    printf("Enter rows and columns of matrix B: ");
    if (scanf("%d%d", &r2, &c2) != 2 || r2 < 1 || c2 < 1 || r2 > 20 || c2 > 20) {
        printf("Invalid dimensions.\n"); return 1;
    }
    a = malloc((size_t)r1 * c1 * sizeof *a);
    b = malloc((size_t)r2 * c2 * sizeof *b);
    result = malloc((size_t)r1 * (c2 > c1 ? c2 : c1) * sizeof *result);
    if (!a || !b || !result) {
        printf("Memory allocation failed.\n");
        free(a); free(b); free(result); return 1;
    }
    printf("Enter matrix A:\n"); read_matrix(a, r1, c1);
    printf("Enter matrix B:\n"); read_matrix(b, r2, c2);
    do {
        printf("\n--- MATRIX OPERATIONS ---\n1. Addition\n2. Subtraction\n3. Multiplication\n4. Exit\nEnter choice: ");
        if (scanf("%d", &choice) != 1) { printf("Invalid choice.\n"); break; }
        switch (choice) {
            case 1:
                if (r1 != r2 || c1 != c2) printf("Addition not possible: dimensions differ.\n");
                else { add_matrices(a, b, result, r1, c1); printf("Sum matrix:\n"); display_matrix(result, r1, c1); }
                break;
            case 2:
                if (r1 != r2 || c1 != c2) printf("Subtraction not possible: dimensions differ.\n");
                else { subtract_matrices(a, b, result, r1, c1); printf("Difference matrix (A-B):\n");
display_matrix(result, r1, c1); }
                break;
            case 3:
                if (c1 != r2) printf("Multiplication not possible: columns of A must equal rows of B.\n");
                else { multiply_matrices(a, b, result, r1, c1, c2); printf("Product matrix:\n"); display_matrix(result,
r1, c2); }
                break;
            case 4: printf("Exiting program.\n"); break;
            default: printf("Invalid choice. Enter 1 to 4.\n");
        }
    } while (choice != 4);
```

```
    free(a); free(b); free(result);
    return 0;
}
```

Compilation and Execution

gcc hdoc71.c -o hdoc71

./hdoc71

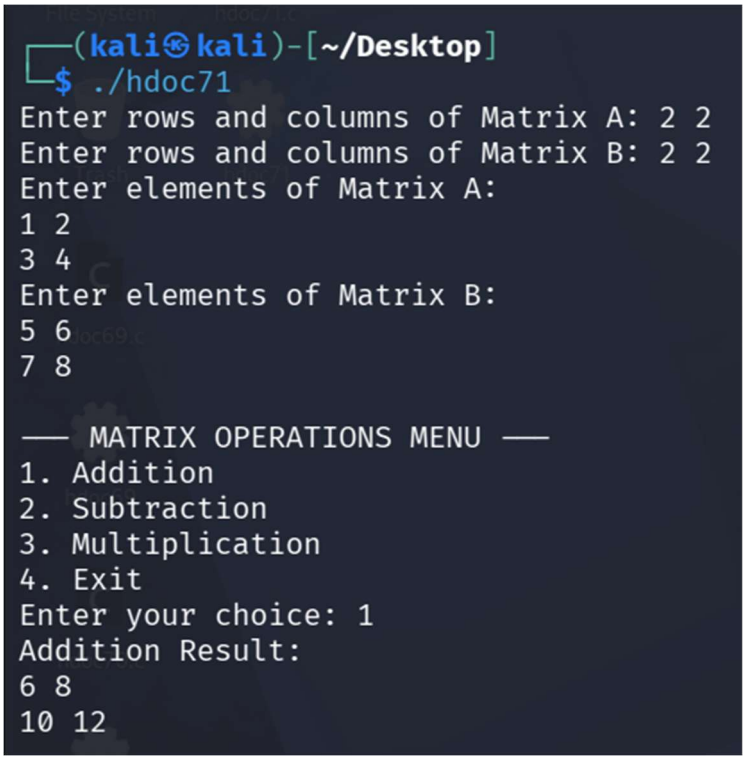
Compilation and Execution



Test Case 1

Input: 2x2 A: 1 2 / 3 4; B: 5 6 / 7 8; choice 1

Expected result: 6 8 / 10 12



Test Case 2

Input: Same matrices; choice 2

Expected result: -4 -4 / -4 -4

```
── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 2
Subtraction Result (A - B):
-4 -4
-4 -4
```

Test Case 3

Input: Same matrices; choice 3

Expected result: 19 22 / 43 50

```
── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 3
Multiplication Result:
19 22
43 50
```

Test Case 4

Input: 1x1 A: 7; B: 3; choice 1

Expected result: 10

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc71
Enter rows and columns of Matrix A: 1 1
Enter rows and columns of Matrix B: 1 1
Enter elements of Matrix A:
7
Enter elements of Matrix B:
3

── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 1
Addition Result:
10
```

Test Case 5

Input: 1x1 A: 7; B: 3; choice 2

Expected result: 4

```
── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 2
Subtraction Result (A - B):
4
```

Test Case 6

Input: 1x1 A: 7; B: 3; choice 3

Expected result: 21

```
── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 3
Multiplication Result:
21
```

Test Case 7

Input: A 2x3: 1 2 3 / 4 5 6; B 3x2: 7 8 / 9 10 / 11 12; choice 3
Expected result: 58 64 / 139 154

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc71
Enter rows and columns of Matrix A: 2 3
Enter rows and columns of Matrix B: 3 2
Enter elements of Matrix A:
1 2 3
4 5 6
Enter elements of Matrix B:
7 8
9 10
11 12

── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 3
Multiplication Result:
58 64
139 154
```

Test Case 8

Input: A 2x2: 1 2 / 3 4; B 2x3: 1 2 3 / 4 5 6; choice 1
Expected result: Addition not possible: dimensions differ.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc71
Enter rows and columns of Matrix A: 2 2
Enter rows and columns of Matrix B: 2 3
Enter elements of Matrix A:
1 2
3 4
Enter elements of Matrix B:
1 2 3
4 5 6

── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 1
Addition not possible.
```

Test Case 9

Input: A 2x3: 1 2 3 / 4 5 6; B 2x2: 1 2 / 3 4; choice 3
Expected result: Multiplication not possible.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc71
Enter rows and columns of Matrix A: 2 3
Enter rows and columns of Matrix B: 2 2
Enter elements of Matrix A:
1 2 3
4 5 6
Enter elements of Matrix B:
1 2
3 4

── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 3
Multiplication not possible.
```

Test Case 10

Input: 1x1 A: 1; B: 1; choice 9, then 4
Expected result: Invalid choice; Exiting program.

```
┌──(kali㉿kali)-[~/Desktop]
└─$ ./hdoc71
Enter rows and columns of Matrix A: 1 1
Enter rows and columns of Matrix B: 1 1
Enter elements of Matrix A:
1
Enter elements of Matrix B:
1

── MATRIX OPERATIONS MENU ──
1. Addition
2. Subtraction
3. Multiplication
4. Exit
Enter your choice: 9
Invalid choice. Enter 1 to 4.
```